

CLOUD SOFT SOLUTIONS · APEX PROGRAM

LangChain Essentials 2026

The 25 Classes That Cover 90% of Real-World AI Applications

A complete beginner-to-advanced guide to the LangChain classes you will actually use in production — what each one is, why it exists, when to reach for it, its core methods, and working Python examples.

TRACK	APEX — Artificial Intelligence & Generative AI
LEVEL	Beginner → Advanced (self-contained)
FORMAT	Trainer handbook with runnable code
AUTHOR	Sreekanth Bathalapalli — Lead Trainer
INSTITUTE	Cloud Soft Solutions, Ameerpet, Hyderabad
EDITION	2026 · Revision 1.0

Contents

Part I — Orientation	6
1. What LangChain Actually Is.....	7
2. The Architecture at a Glance.....	9
Part II — The Model Layer	11
3. ChatOpenAI — Calling the Model.....	12
4. ChatPromptTemplate — Reusable Prompts.....	16
5. SystemMessage, HumanMessage, AIMessage.....	19
Part III — Shaping the Output	22
6. StrOutputParser.....	23
7. JsonOutputParser.....	25
8. Structured Output with Pydantic.....	27
Part IV — LCEL and the Runnables	30
9. LCEL — LangChain Expression Language.....	31
10. RunnableLambda.....	34
11. RunnableParallel.....	36
12. RunnablePassthrough.....	38
Part V — Retrieval and RAG	40
13. Document Loaders.....	41
14. Text Splitters.....	44
15. Embeddings.....	47
16. Vector Stores.....	50
17. Retrievers.....	53
18. RetrievalQA — the Classic Chain.....	55
19. The End-to-End RAG Pipeline.....	58
Part VI — Tools and Agents	61
20. Tools.....	62
21. Tool Calling.....	64
22. Agents.....	66
23. AgentExecutor.....	68
Part VII — Memory and Conversation State	71
24. Conversation Memory — Legacy and Modern.....	72
25. Message History in Practice.....	75
Part VIII — Reference and Next Steps	77
26. Production Class Reference.....	78
27. A 21-Day Study Plan.....	80
Appendix A — Troubleshooting.....	82
Appendix B — Glossary.....	83
About Cloud Soft Solutions.....	84

How to Use This Handbook

LangChain is large. The API surface runs to several hundred classes across a dozen packages, and every release moves something. That size scares beginners into one of two failure modes: they either try to memorise the whole library, or they copy a tutorial without understanding a single line of it.

Neither works. The way through is much narrower than it looks.

The 25-class rule

In production systems — ChatGPT clones, RAG pipelines, document Q&A, customer-support bots, coding assistants, enterprise agents — roughly **25 to 30 classes** do almost all the work. Learn those properly and you can read, write, and debug **about 90% of real LangChain code** you will ever meet.

Everything else is a variation on the same handful of ideas: a **Runnable** that takes input, does one job, and returns output.

What each chapter gives you

- **Why it exists** — the problem the class was invented to solve. Skip this and you will use the class in the wrong place.
- **When to use it** — the signal in your own requirement that tells you "this is the one".
- **The code** — short, runnable, no framework noise.
- **The output** — what you should actually see on screen, so you know when it worked.
- **Methods that matter** — a compact table of the methods used in real projects, not the full API dump.
- **Real-world use case** — where this appears in a shipped application.

How to work through it

1. Read Part I end to end. It is short and it makes everything after it make sense.
2. For Parts II to VII, **type the code**. Do not paste it. Typing forces you to notice the import paths, which are the single most common source of beginner errors in LangChain.
3. After each chapter, break the example on purpose. Remove a variable from the prompt. Pass the wrong type. Read the traceback. That is where the learning is.
4. Build the Part V end-to-end RAG pipeline over one of your own documents before moving to agents.
5. Only then go to Part VIII and LangGraph.

What you need installed

TERMINAL

```
# Core + OpenAI provider
pip install langchain langchain-core langchain-openai

# Community integrations (loaders, older vector stores)
pip install langchain-community

# Splitters and a local vector store
pip install langchain-text-splitters faiss-cpu
```

```
# Structured output
pip install pydantic

# Your key
export OPENAI_API_KEY="sk-..."
```

A note on package names

LangChain was split into separate packages. `langchain_core` holds the primitives (messages, prompts, parsers, runnables). `langchain_openai`, `langchain_anthropic`, `langchain_google_genai` and friends hold the provider bindings. `langchain_community` holds third-party integrations. `langchain` itself is now mostly high-level chains and agent helpers. **Most import errors beginners hit are because they used the old flat `from langchain import ...` path.**

Provider independence

Every example in this handbook uses `ChatOpenAI` because it is the most common starting point and the easiest to get a key for. The important thing is that **the rest of the code does not care**. Swap the two lines that create the model and everything downstream — prompts, parsers, chains, retrievers, agents — keeps working unchanged.

SAME INTERFACE, FOUR PROVIDERS

```
# OpenAI
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4.1", temperature=0.2)

# Anthropic
from langchain_anthropic import ChatAnthropic
llm = ChatAnthropic(model="claude-sonnet-4-5", temperature=0.2)

# Google Gemini
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0.2)

# Local via Ollama
from langchain_ollama import ChatOllama
llm = ChatOllama(model="llama3.1")
```

That interchangeability **is** the reason LangChain exists. Hold on to it — it is the thread running through the entire book.

PART I

Orientation

Before a single class: what LangChain is for, the shape of every application you will build with it, and the one abstraction that ties the whole library together.

1 What LangChain Actually Is

Strip away the marketing and LangChain is three things:

6. A **uniform interface** over models, so your application code does not change when the model does.
7. A **set of small composable pieces** — prompts, parsers, retrievers, tools — that all speak the same protocol.
8. A **wiring language** (LCEL) for connecting those pieces into a pipeline with one operator.

That is it. Everything in this handbook is an instance of one of those three ideas.

The problem it solves

Write an LLM application without a framework and it starts out beautifully simple:

RAW SDK — DAY ONE

```
import openai

response = openai.chat.completions.create(
    model="gpt-4.1",
    messages=[{"role": "user", "content": "Explain Kubernetes"}]
)
print(response.choices[0].message.content)
```

Six weeks later that same file has grown to nine hundred lines, because the requirements grew:

- "Also support Gemini, the OpenAI bill is too high." — every call site now needs a branch, and the response shape is different.
- "Answer from our PDFs, not from training data." — you now need loading, chunking, embedding, storage, and search.
- "Return JSON so we can insert it into Postgres." — you are now writing regex against model prose.
- "Let it call our internal APIs." — you are hand-rolling a tool-dispatch loop.
- "Stream it, the UI feels dead." — a second code path for every one of the above.

Each of those is a solved problem. LangChain is the library of solutions, with the pieces designed to snap together.

The one abstraction: Runnable

Nearly every class you meet in this book implements a single interface called **Runnable**. A Runnable is anything that accepts an input, does one job, and returns an output. Because they all share that shape, they all share the same four methods:

Method	What it does	When you use it
<code>invoke(x)</code>	Run once, synchronously, return the result	Almost always — scripts, APIs, testing
<code>batch([x, y, z])</code>	Run many inputs concurrently	Bulk jobs — 500 product descriptions
<code>stream(x)</code>	Yield the result in pieces as it is produced	Chat UIs, anything a human is watching
<code>ainvoke</code> / <code>abatch</code> / <code>astream</code>	Async versions of the above	FastAPI, async workers, high concurrency

This is the single most useful thing to internalise

A prompt template is a Runnable. A model is a Runnable. An output parser is a Runnable. A retriever is a Runnable. A plain Python function wrapped in `RunnableLambda` is a Runnable. **A chain of Runnables is itself a Runnable.**

So once you know `invoke`, `batch` and `stream`, you already know how to drive every component in the library — and every pipeline you assemble out of them.

What LangChain is not

- **Not a model.** It calls models. You still need an API key or a local runtime.
- **Not required.** Small, single-provider scripts are often cleaner with the raw SDK. Reach for LangChain when you have retrieval, tools, multiple providers, or composition.
- **Not the state machine.** Long-running, branching, multi-step agent workflows belong in **LangGraph**, its sibling library. LangChain gives you the components; LangGraph gives you the control flow.

2 The Architecture at a Glance

Almost every LangChain application is one of two shapes, and both start the same way. Keep this map in your head — when you get lost in a codebase, find where you are on it.

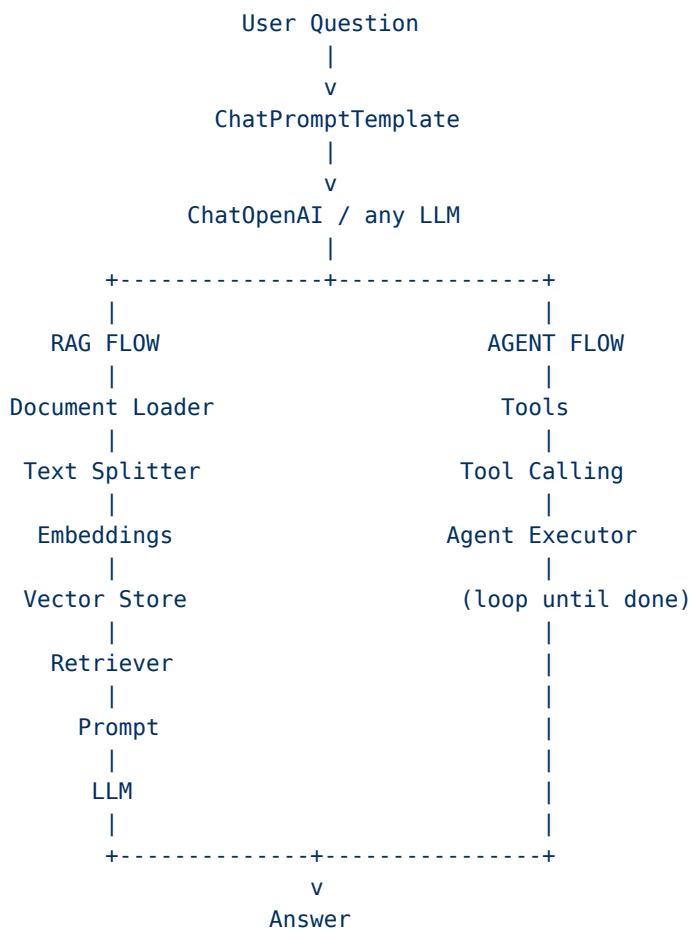


Figure 2.1 — The two canonical LangChain application shapes.

RAG flow — "answer from my documents"

Retrieval-Augmented Generation. The model does not know your company handbook, your product catalogue, or last quarter's incident reports. RAG solves that by **finding the relevant passages first and pasting them into the prompt**. The model then answers from text you supplied rather than from memory.

Use it when the requirement contains the words **our**, **my**, **internal**, **policy**, **manual**, **documentation**, or **knowledge base**.

Agent flow — "do something for me"

The model is given a set of tools — Python functions, API calls, database queries — and decides for itself which to call and in what order. It thinks, acts, observes the result, and loops until it can answer.

Use it when the requirement contains **look up**, **calculate**, **book**, **create a ticket**, **check the status of**, or any verb that touches the outside world.

The two are not exclusive

Serious applications combine them: an agent that has a retriever as one of its tools. A support bot searches the knowledge base (RAG), and if that fails, opens a Zendesk ticket (agent). Build them separately, then join.

Where each chapter of this book sits on the map

Layer	Classes	Part
Model interface	ChatOpenAI , message classes	Part II
Prompting	ChatPromptTemplate	Part II
Output shaping	StrOutputParser , JsonOutputParser , with_structured_output	Part III
Wiring	LCEL , RunnableLambda , RunnableParallel , RunnablePassthrough	Part IV
Retrieval	Loaders, splitters, embeddings, vector stores, retrievers	Part V
Action	@tool , bind_tools , agents, AgentExecutor	Part VI
State	Memory, message history	Part VII

PART II

The Model Layer

Calling the model, shaping what you send it, and the three message types that control who is speaking.

3 ChatOpenAI — Calling the Model

The most-used class in the library. It is the object that actually talks to the LLM. Every other class in this book exists to prepare its input or process its output.

Why it exists

To give every chat model on earth the same three methods. Whether you are calling GPT, Claude, Gemini, Mistral or a Llama running on your own laptop, you call `invoke`, `batch`, `stream` — and you get back the same message object. Provider differences stop at this boundary.

Creating one

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4.1",
    temperature=0.2
)
```

The constructor arguments that matter

Argument	Meaning	Practical guidance
<code>model</code>	Which model to call	Pin it explicitly. Never rely on the default.
<code>temperature</code>	Randomness, 0.0 – 2.0	<code>0</code> for extraction, classification, SQL. <code>0.2</code> for factual explanation. <code>0.7+</code> for marketing copy.
<code>max_tokens</code>	Cap on the response length	Set it in production. It is your cost ceiling.
<code>timeout</code>	Seconds before giving up	Always set. Default hangs are a real outage cause.
<code>max_retries</code>	Automatic retry count	Default 2. Raise for flaky networks.
<code>api_key</code>	Your key	Omit it — read <code>OPENAI_API_KEY</code> from the environment instead.

Never hardcode the key

Putting `api_key="sk-..."` in source is how keys end up in Git history and then on GitHub. Use environment variables, a `.env` file loaded with `python-dotenv`, or your cloud provider's secret manager. Add `.env` to `.gitignore` before you write a single line.

Method 1 — `invoke()`

Runs one request and waits for the whole answer.

```
response = llm.invoke("What is Kubernetes?")
print(response.content)
```

OUTPUT

Kubernetes is a container orchestration platform that automates the deployment, scaling, and management of containerised applications...

.content is not optional

`invoke()` returns an **AIMessage object**, not a string. Print it directly and you will see the whole wrapper — metadata, token counts, tool calls and all. The text lives in `.content`. Forgetting this is the single most common beginner error.

What else is on the response

```
r = llm.invoke("What is Kubernetes?")

print(r.content)           # the text
print(r.response_metadata) # model name, finish reason
print(r.usage_metadata)    # input_tokens, output_tokens, total_tokens
print(r.id)                # provider-side message id
```

`usage_metadata` is how you build cost dashboards. Log it from day one — retrofitting billing telemetry into a live system is miserable.

Method 2 — batch()

Send many prompts and get many answers, processed concurrently rather than one after another.

```
questions = [
    "What is AWS?",
    "What is Azure?",
    "What is GCP?"
]

responses = llm.batch(questions)

for r in responses:
    print(r.content)
    print("-" * 40)
```

Results come back **in the same order as the inputs**, always. You can safely `zip()` them against the source list.

Controlling concurrency

```
responses = llm.batch(
    questions,
    config={"max_concurrency": 5}
)
```

Without a limit you will hit provider rate limits on large lists. Five to ten is a sane starting point.

Surviving partial failures

```
responses = llm.batch(questions, return_exceptions=True)

for q, r in zip(questions, responses):
    if isinstance(r, Exception):
        print(f"FAILED: {q} -> {r}")
    else:
        print(r.content)
```

Without `return_exceptions=True`, one bad input kills the entire batch and you lose the 499 answers that succeeded.

Real-world use case

A catalogue team needs product descriptions for 4,000 SKUs. A loop with `invoke()` takes hours. `batch()` with `max_concurrency=8` finishes in minutes, at identical cost.

Method 3 — `stream()`

Yields the answer token by token as the model produces it, instead of waiting for the whole thing.

```
for chunk in llm.stream("Explain Docker"):
    print(chunk.content, end="", flush=True)
```

OUTPUT

```
Docker is a platform for building, shipping, and running
applications inside lightweight containers...
```

(printed progressively, a few characters at a time)

Each `chunk` is an `AIMessageChunk`, not a string — `.content` again. `flush=True` matters in terminals, otherwise Python buffers and you lose the effect entirely.

Why it matters more than it looks

Streaming does not make the model faster. It makes it **feel** faster, which in a user-facing product is the same thing. Total time to the last token is unchanged; time to the **first** token drops from ten seconds to a few hundred milliseconds, and the user stops wondering whether the page has frozen. This is exactly why every chat interface you have ever used streams.

Streaming from a web API

FASTAPI + ASTREAM

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

@app.get("/ask")
```

```
async def ask(q: str):
    async def gen():
        async for chunk in llm.astream(q):
            yield chunk.content
    return StreamingResponse(gen(), media_type="text/plain")
```

Choosing between the three

Situation	Use	Reason
One question, script or backend job	<code>invoke()</code>	Simplest; you need the whole answer anyway
Many independent inputs	<code>batch()</code>	Concurrency without writing thread pools
A human is watching a screen	<code>stream()</code>	Perceived latency collapses
Inside FastAPI / async worker	<code>ainvoke()</code> / <code>astream()</code>	Does not block the event loop

4 ChatPromptTemplate — Reusable Prompts

A prompt with named holes in it, that you fill at call time.

Why it exists

You will write this first:

```
prompt = f"Explain {topic}"
```

And it works, until it does not. F-strings break down for four specific reasons:

- **They evaluate immediately.** You cannot define the prompt in a config file, ship it to a template store, or hand it to a non-engineer to edit.
- **They fail silently on typos.** `f"Explain {topc}"` is a `NameError` at runtime, in production, at 2am.
- **They cannot be composed.** A template is a `Runnable`; an f-string is a string. Only one of them can be piped into a model.
- **They do not carry role structure.** Real prompts are a system message plus a human message plus history — an f-string is one flat blob.

The basic form

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple words."
)

formatted = prompt.invoke({"topic": "Kubernetes"})
print(formatted)
```

OUTPUT

```
messages=[HumanMessage(content='Explain Kubernetes in simple words.')
```

Note what came back: not a string, but a **list of messages** — already in the shape the model wants. That is why the template can be piped straight into the LLM.

Multiple variables

```
prompt = ChatPromptTemplate.from_template("""
Explain {topic}
Level      : {level}
Language   : {language}
""")

prompt.invoke({
    "topic": "AWS",
    "level": "Beginner",
    "language": "English"
})
```

Every placeholder must be supplied. Miss one and you get a clear `KeyError` naming the missing variable — which is precisely the failure mode f-strings deny you.

The form you should actually use — `from_messages()`

`from_template()` produces a single human message. Real applications separate the instructions from the question, because that is what gives you reliable control over behaviour.

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an AWS trainer. Answer in {language}. "
               "Use short paragraphs and a concrete example."),
    ("human", "{question}")
])

msgs = prompt.invoke({
    "language": "English",
    "question": "What is an EC2 instance?"
})
```

The system message is where your product's personality, tone, refusal rules, and output format live. Keeping it separate means you can change the persona without touching the question handling.

Leaving room for conversation history

```
from langchain_core.prompts import MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful DevOps assistant."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{question}")
])

prompt.invoke({
    "history": [
        HumanMessage(content="My name is Sreekanth"),
        AIMessage(content="Nice to meet you, Sreekanth!")
    ],
    "question": "What is my name?"
})
```

`MessagesPlaceholder` is the hinge every multi-turn chatbot swings on. It is a slot for a whole list of past messages, dropped into the middle of the prompt.

`partial()` — locking in values early

```
base = ChatPromptTemplate.from_template(
    "You are a {role}. Answer in {language}. Question: {question}"
)

telugu_trainer = base.partial(role="DevOps trainer", language="Telugu")

telugu_trainer.invoke({"question": "What is Jenkins?"})
```

Useful when part of the prompt is fixed per tenant, per user, or per deployment, and only the last variable changes per request.

Escaping literal braces

The JSON trap

If your prompt contains a JSON example, its braces will be read as variables and blow up. Double them to escape:

Return JSON like `{{"name": "value"}}` for the topic `{topic}` — the doubled braces render as single ones, and `{topic}` stays a variable.

Methods that matter

Method	Purpose
<code>from_template(str)</code>	Quick single-message prompt
<code>from_messages([...])</code>	System + human + placeholders — the production form
<code>invoke(dict)</code>	Fill the variables, return messages
<code>partial(**kwargs)</code>	Pre-fill some variables, return a new template
<code>input_variables</code>	Property listing every placeholder — good for validation

Real-world use case

One template, four products. A bank chatbot, an HR bot, a medical triage assistant and a customer-support agent can share identical pipeline code and differ only in the system message stored in the database. Prompts become configuration, not code.

5 SystemMessage, HumanMessage, AIMessage

Three tiny classes that carry the single most important piece of information in a conversation: **who said it**.

Why they exist

Chat models are not trained on flat text. They are trained on a transcript with speakers labelled. The label changes how the content is weighted: instructions in the system role are treated as standing rules, text in the human role is treated as a request, text in the AI role is treated as something the model itself already committed to.

Send the same words in the wrong role and you get a materially different answer. These classes are how you set the role.

SystemMessage — the standing instruction

```
from langchain_core.messages import SystemMessage

system = SystemMessage(
    content="You are an AWS Trainer."
)
```

It goes first, appears once, and governs the whole conversation. This is where you put persona, tone, output format, domain limits and refusal rules.

A PRODUCTION-GRADE SYSTEM MESSAGE

```
system = SystemMessage(content="""
You are a senior AWS trainer at Cloud Soft Solutions.

Rules:
- Explain concepts in plain language before naming services.
- Always give one concrete, runnable example.
- If the question is not about cloud computing, say so and stop.
- Never invent pricing figures. Direct users to the AWS pricing page.
""")
```

HumanMessage — the user turn

```
from langchain_core.messages import HumanMessage

human = HumanMessage(content="Explain EC2.")
```

Whatever came from the person, the form, or the upstream API. **Treat its content as untrusted** — it is where prompt injection arrives.

AIMessage — the assistant turn

```
from langchain_core.messages import AIMessage

ai = AIMessage(content="EC2 is a virtual server in the cloud...")
```

You get these back from `invoke()`. You also **construct them by hand** when replaying conversation history, so the model can see what it said before.

Putting them together

```
messages = [
    SystemMessage(content="You are a DevOps Trainer."),
    HumanMessage(content="Explain Docker")
]

response = llm.invoke(messages)
print(response.content)
```

A multi-turn conversation is just a longer list, alternating human and AI:

```
messages = [
    SystemMessage(content="You are a DevOps Trainer."),
    HumanMessage(content="Explain Docker"),
    AIMessage(content="Docker packages an application and its "
                    "dependencies into a container image..."),
    HumanMessage(content="Now compare it to a virtual machine.")
]

response = llm.invoke(messages)
```

Models have no memory

This is the fact that surprises everyone. Each API call is completely independent — the model remembers nothing between requests. A chatbot "remembers" only because your application **resends the entire message list every single time**. Memory is your job, not the model's. Chapters 22 and 23 cover how to manage it.

The other message types

Class	Role	Where it appears
<code>SystemMessage</code>	Standing instructions	First message, once
<code>HumanMessage</code>	User input	Every user turn
<code>AIMessage</code>	Model output	Returned by <code>invoke()</code> ; replayed as history
<code>ToolMessage</code>	Result of a tool call	Agent loops (Part VI)
<code>AIMessageChunk</code>	A streamed fragment	Returned by <code>stream()</code>

The tuple shorthand

LangChain accepts `(role, content)` tuples wherever messages are expected, which is shorter for static prompts:

```
llm.invoke([
    ("system", "You are a DevOps Trainer."),
    ("human", "Explain Docker")
])
```

Use tuples inside `ChatPromptTemplate.from_messages()`. Use the real classes when you are building history lists programmatically, because the type checking helps.

PART III

Shaping the Output

Models return prose. Applications need strings, JSON and typed objects. Three classes close that gap.

6 StrOutputParser

Takes the message object off the model and hands you the plain string inside it.

Why it exists

Without it, every chain ends with a wrapper you have to unwrap:

WITHOUT A PARSER

```
AIMessage(content='Linux is an operating system...',
          response_metadata={...}, usage_metadata={...})
```

With it, the chain ends with what you wanted in the first place:

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()
chain = llm | parser

result = chain.invoke("What is Linux?")
print(result)
```

OUTPUT

```
Linux is an open-source operating system kernel first released
in 1991 by Linus Torvalds...
```

`result` is now a genuine Python `str`. You can slice it, `len()` it, store it in a database column, or return it from an API without a second thought.

Why it is more than convenience

- **It makes the chain terminate in a usable type.** Anything downstream — a template, a function, a response model — expects a string, not a message.
- **It preserves streaming.** `StrOutputParser` is stream-aware: pipe it after a model and `chain.stream()` still yields text pieces as they arrive.
- **It normalises across providers.** Different providers return slightly different content shapes; the parser flattens them all to text.

STREAMING SURVIVES THE PARSER

```
chain = prompt | llm | StrOutputParser()

for piece in chain.stream({"topic": "Kubernetes"}):
    print(piece, end="", flush=True)
```

Rule of thumb

If the end consumer of the chain is a human reading text, end with `StrOutputParser()`. If the end consumer is code that needs fields, use one of the next two chapters instead.

7 JsonOutputParser

Parses the model's answer into a Python dictionary.

Why it exists

Sooner or later the output stops being for a person and starts being for a database, an API, or a downstream service. At that point prose is a liability — you need fields.

The naive approach is to ask for JSON in the prompt and then hope. The model wraps it in a code fence, adds "Here is the JSON you requested:", or emits a trailing comma, and your `json.loads()` explodes.

`JsonOutputParser` handles the unwrapping and the common malformations for you.

Basic use

```
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import ChatPromptTemplate

parser = JsonOutputParser()

prompt = ChatPromptTemplate.from_template(
    "Give the cloud provider and its compute service for {topic}. "
    "Respond only with JSON.\n{format_instructions}"
).partial(format_instructions=parser.get_format_instructions())

chain = prompt | llm | parser

result = chain.invoke({"topic": "AWS"})
print(result)
print(result["service"])
```

OUTPUT

```
{'name': 'AWS', 'service': 'EC2'}
EC2
```

`get_format_instructions()` is the important part

The parser can tell the model exactly what shape it expects. Injecting those instructions into the prompt is what turns "usually works" into "reliably works". Skip it and you are back to hoping.

Constraining the schema

An empty `JsonOutputParser` accepts any JSON at all — including JSON with the wrong keys. Give it a Pydantic model and it enforces structure:

```
from pydantic import BaseModel, Field
from langchain_core.output_parsers import JsonOutputParser

class CloudService(BaseModel):
    provider: str = Field(description="Cloud provider name")
    service: str = Field(description="Primary compute service")
    region_example: str = Field(description="One example region code")
```

```
parser = JsonOutputParser(pydantic_object=CloudService)
```

The `description` on each field is not documentation for you — it is shipped to the model as part of the format instructions. Write them as instructions, not as notes.

Where this lands in real systems

Application	What gets extracted
Invoice processing	vendor, invoice number, line items, total, due date
Resume screening	name, years of experience, skills array, current CTC
Support ticket triage	category, severity, product area, suggested queue
Log analysis	error type, affected service, probable root cause
Lead capture from email	company, contact, budget, timeline, intent score

When `JsonOutputParser` is not enough

It returns a `dict`. Dicts are untyped — `result["provdier"]` fails at runtime with a `KeyError`, not at edit time. For anything where correctness matters, use `with_structured_output()` from Chapter 8 instead, which gives you a validated object.

8 Structured Output with Pydantic

The modern, provider-native way to get typed objects out of a model. If you learn one thing from Part III, learn this.

Why it exists

Parsers work *“after the fact”* — the model produces text, and you try to rescue structure from it. Structured output works *“before the fact”*: the schema is sent to the provider as part of the request, and the provider constrains generation so the response cannot be malformed.

The difference in reliability is not marginal. It is the difference between a demo and a system you can put in front of paying customers.

The pattern

```
from pydantic import BaseModel, Field

class Employee(BaseModel):
    name: str = Field(description="Full name of the employee")
    department: str = Field(description="Department they work in")

structured_llm = llm.with_structured_output(Employee)

employee = structured_llm.invoke("Generate an employee record.")
print(employee)
print(employee.name)
```

OUTPUT

```
name='Priya Sharma' department='Engineering'
Priya Sharma
```

`employee` is a real `Employee` instance. Your editor autocompletes `.name`. A typo is caught before you run. Pydantic has already validated the types.

A realistic schema

```
from typing import Literal, Optional
from pydantic import BaseModel, Field

class Ticket(BaseModel):
    """A triaged customer support ticket."""

    summary: str = Field(description="One-sentence summary of the issue")
    category: Literal["billing", "technical", "account", "other"] = Field(
        description="Best-fit category"
    )
    severity: Literal["low", "medium", "high", "critical"] = Field(
        description="Business impact of the issue"
    )
    affected_service: Optional[str] = Field(
        default=None,
        description="Product or service named by the customer, if any"
```

```
)
needs_human: bool = Field(
    description="True if this requires a human agent"
)

triage = llm.with_structured_output(Ticket)

t = triage.invoke(
    "Our production database has been down for 40 minutes and "
    "we cannot process any orders. This is urgent."
)

print(t.severity, "|", t.category, "|", t.needs_human)
```

OUTPUT

```
critical | technical | True
```

Three things that make schemas work well

- 9. The docstring is a prompt.** The class docstring is sent to the model. Use it to describe what the object represents.
- 10. `Literal` beats `str` every time.** Constraining a category to four options removes an entire class of failure. The model cannot invent a fifth.
- 11. Every field gets a `description`.** Field names alone are ambiguous; `severity` means nothing without a definition of the scale.

Nested and repeated structures

```
from typing import List

class LineItem(BaseModel):
    description: str
    quantity: int
    unit_price: float

class Invoice(BaseModel):
    """Data extracted from a supplier invoice."""
    vendor: str
    invoice_number: str
    items: List[LineItem] = Field(description="Every line on the invoice")
    total: float

extractor = llm.with_structured_output(Invoice)
invoice = extractor.invoke(raw_invoice_text)

for item in invoice.items:
    print(item.description, item.quantity, item.unit_price)
```

Nesting and lists work exactly as you would expect. This one pattern replaces most document-extraction code people used to write by hand.

Parser versus structured output

	<code>`JsonOutputParser`</code>	<code>`with_structured_output()`</code>
Returns	Python <code>dict</code>	Typed Pydantic object
Validation	Optional, after the fact	Enforced by Pydantic
Schema sent to model	As text in the prompt	Natively, via the provider API
Reliability	Good	Very high
Editor support	None	Full autocomplete and type checks
Streaming	Partial dicts supported	Limited
Use when	Provider lacks native support	Default choice in 2026

Real-world use case

HR automation parsing 900 resumes into a database. CRM enrichment from inbound emails. Invoice extraction for finance. Any place where a human currently retypes information from a document into a form is a candidate for `with_structured_output()`.

PART IV

LCEL and the Runnables

One operator to connect everything, and three small classes that let arbitrary Python live inside a chain.

9 LCEL — LangChain Expression Language

The pipe operator. The reason modern LangChain code is short.

Why it exists

Before LCEL, connecting components meant explicit chain classes and a lot of glue:

MANUAL WIRING

```
# The old way – verbose, hard to modify
formatted = prompt.invoke({"topic": "Python"})
response = llm.invoke(formatted)
text = parser.invoke(response)
```

LCEL collapses that into one declaration:

LCEL

```
chain = prompt | llm | parser

result = chain.invoke({"topic": "Python"})
```

Read it left to right: the input dictionary goes into the prompt, the prompt's messages go into the model, the model's message goes into the parser, and a string comes out.

What you get for free

The composed chain is itself a Runnable, so it inherits the entire interface without you writing a line:

```
chain.invoke({"topic": "Python"})           # one result
chain.batch([{"topic": "Go"}, {"topic": "Rust"}]) # concurrent
chain.stream({"topic": "Python"})           # token by token
await chain.ainvoke({"topic": "Python"})     # async
```

This is the payoff

You wrote three components and one pipe. You received synchronous execution, concurrent batching, streaming, async, retries, fallbacks, and tracing — on the whole pipeline, not just on the model. That is what "composable" actually buys you.

A pipeline that ends in a database

```
Input dict
  |
  v
ChatPromptTemplate  fills the variables
  |
  v
ChatOpenAI         calls the model
  |
  v
```

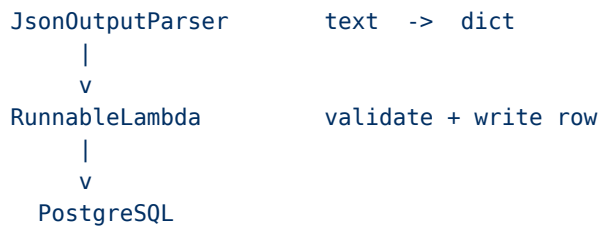


Figure 9.1 — Every stage is a Runnable, so the whole thing is one object.

Adding resilience

```

# Retry the whole chain on transient failure
robust = chain.with_retry(stop_after_attempt=3)

# Fall back to a cheaper model if the primary fails
from langchain_openai import ChatOpenAI
backup = prompt | ChatOpenAI(model="gpt-4.1-mini") | parser
resilient = chain.with_fallbacks([backup])

# Attach configuration for tracing and limits
result = chain.invoke(
    {"topic": "Python"},
    config={"max_concurrency": 4, "tags": ["prod", "explainer"]}
)

```

Branching mid-chain

```

from langchain_core.runnables import RunnableBranch

route = RunnableBranch(
    (lambda x: "error" in x["question"].lower(), troubleshooting_chain),
    (lambda x: "price" in x["question"].lower(), pricing_chain),
    general_chain          # default
)

```

[RunnableBranch](#) is a router: the first condition that matches decides which sub-chain runs. This is how a single endpoint serves several intents.

Debugging a chain

```

# See the input/output of every step
from langchain.globals import set_debug
set_debug(True)

# Or inspect the structure
print(chain.get_graph().draw_ascii())

```

When a chain misbehaves, the fault is almost always a **shape mismatch** — one step returns a dict where the next expects a string. [set_debug\(True\)](#) shows you exactly where.

10 RunnableLambda

Wraps an ordinary Python function so it can sit inside a chain.

Why it exists

Not everything is a model call. Real pipelines need to clean input, strip whitespace, validate a field, look something up in a database, format a date, redact a phone number. Those are plain Python. [RunnableLambda](#) is the adapter that lets plain Python join the pipe.

Basic use

```
from langchain_core.runnables import RunnableLambda

def uppercase(text):
    return text.upper()

upper = RunnableLambda(uppercase)

print(upper.invoke("cloud"))
```

OUTPUT

CLOUD

Inside a chain

```
def clean(text: str) -> str:
    return text.strip().replace("\n", " ")

def add_disclaimer(text: str) -> str:
    return text + "\n\n- Generated by Cloud Soft Solutions AI Assistant"

chain = (
    RunnableLambda(clean)
    | prompt
    | llm
    | StrOutputParser()
    | RunnableLambda(add_disclaimer)
)
```

Pre-processing before the model, post-processing after it. Both are just functions.

The automatic conversion

Inside a chain that already contains a Runnable, LangChain wraps bare functions for you:

```
# These two are equivalent
chain = prompt | llm | StrOutputParser() | RunnableLambda(add_disclaimer)
chain = prompt | llm | StrOutputParser() | add_disclaimer
```

Be explicit anyway when the chain is long — it reads better and it makes the intent obvious to whoever maintains it.

The one-argument rule

A function in a chain takes exactly one argument

The output of the previous step is passed as a single value. If you need more, take a dict:

```
def render(d): return f"{d['name']} works in {d['dept']}"
```

And if you need to bind extra static arguments, use `functools.partial` before wrapping.

Practical uses

Stage	Example function
Input cleaning	strip whitespace, normalise case, collapse newlines
Validation	reject empty questions; enforce a length limit
Redaction	mask card numbers and phone numbers before they reach the model
Enrichment	look the user up in the CRM and attach their tier
Formatting	join retrieved documents into one context string
Persistence	write the parsed result to Postgres and return the row id
Guardrails	run a profanity or PII check on the answer before returning it

That fifth row — joining retrieved documents — is the one you will write in every RAG project. Chapter 20 uses it.

11 RunnableParallel

Runs several branches at the same time on the same input, and returns a dictionary of their results.

Why it exists

If a document needs a summary, a keyword list, a translation and a sentiment score, running them one after another costs four round trips in series. They do not depend on each other, so there is no reason to wait.

[RunnableParallel](#) fires them together and you pay for the slowest one, not the sum.

Basic use

```
from langchain_core.runnables import RunnableParallel

chain = RunnableParallel({
    "summary": llm,
    "keywords": llm
})

result = chain.invoke("Explain Kubernetes")
print(result["summary"].content)
```

The version you will actually write

Pointing two branches at the same bare model gives you two similar answers. In practice each branch is its own small chain with its own prompt:

```
summary_chain = (
    ChatPromptTemplate.from_template("Summarise in two sentences:\n{text}")
    | llm | StrOutputParser()
)

keyword_chain = (
    ChatPromptTemplate.from_template(
        "List 5 keywords as a comma-separated line:\n{text}"
    )
    | llm | StrOutputParser()
)

sentiment_chain = (
    ChatPromptTemplate.from_template(
        "Reply with exactly one word – positive, neutral or negative:\n{text}"
    )
    | llm | StrOutputParser()
)

analyse = RunnableParallel(
    summary=summary_chain,
    keywords=keyword_chain,
    sentiment=sentiment_chain
)

result = analyse.invoke({"text": article})
print(result["sentiment"])
```

OUTPUT

```
{
  'summary': 'The article explains how container orchestration...',
  'keywords': 'kubernetes, containers, orchestration, scaling, pods',
  'sentiment': 'neutral'
}
```

One input in, a labelled dictionary out. Three API calls, one wall-clock wait.

The dict shorthand

Anywhere a Runnable is expected, a plain dictionary of Runnables is automatically treated as a [RunnableParallel](#):

```
chain = {"summary": summary_chain, "keywords": keyword_chain} | combiner
```

You will see this constantly in RAG code, and Chapter 12 explains why.

Where it pays off

- **Document analysis dashboards** — summary, entities, sentiment and risk score rendered together.
- **Multilingual publishing** — one article fanned out to Telugu, Hindi and English simultaneously.
- **Multi-retriever RAG** — query a vector store, a SQL database and a web search at once, then merge.
- **Model comparison** — send the same prompt to three providers and score the answers side by side.
- **Content pipelines** — generate a title, a meta description and a social caption from one draft.

12 RunnablePassthrough

Returns its input unchanged. Which sounds pointless, until you build your first RAG chain.

Basic behaviour

```
from langchain_core.runnables import RunnablePassthrough

chain = RunnablePassthrough()
print(chain.invoke("AWS"))
```

OUTPUT

AWS

Why an identity function is useful

In a parallel step, every branch consumes the same input and transforms it. But often one value must survive **untransformed** all the way to the end. The classic case is RAG: the retriever turns the question into documents, and the prompt still needs the original question.

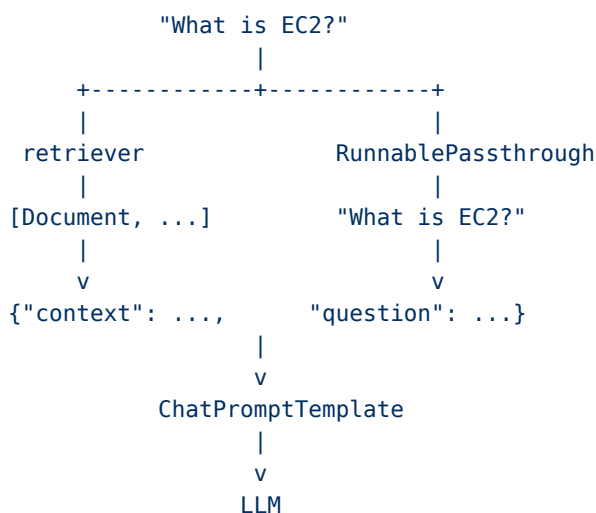


Figure 12.1 — The passthrough is how the original question reaches the prompt.

The canonical RAG wiring

```
from langchain_core.runnables import RunnablePassthrough

rag_chain = (
    {"context": retriever | format_docs,
     "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

rag_chain.invoke("What is Amazon EC2?")
```

The dictionary is an implicit `RunnableParallel`. Both keys receive the same string. The `context` key routes it through retrieval; the `question` key hands it straight on. The prompt receives exactly the two variables it declared. **Memorise this block** — you will write it in every retrieval project you ever build.

assign() — add a key, keep the rest

```
chain = RunnablePassthrough.assign(
    context=lambda x: retriever.invoke(x["question"])
)

# in: {"question": "What is EC2?"}
# out: {"question": "What is EC2?", "context": [Document, ...]}
```

`assign()` is the accumulating version: it keeps everything already in the dictionary and adds one more computed key. Use it when a pipeline builds up state across several steps.

Multi-stage accumulation

```
pipeline = (
    RunnablePassthrough.assign(context=retrieve_step)
    | RunnablePassthrough.assign(answer=answer_step)
    | RunnablePassthrough.assign(citations=citation_step)
)

# Final dict carries question, context, answer AND citations
```

This is how you return an answer **with** its sources — a hard requirement in every enterprise deployment, because nobody signs off on an AI answer they cannot trace.

The three runnables side by side

Class	Takes	Returns	Use it for
<code>RunnableLambda</code>	One value	Whatever your function returns	Custom Python inside a chain
<code>RunnableParallel</code>	One value	A dict of branch results	Independent work run concurrently
<code>RunnablePassthrough</code>	One value	The same value	Carrying the original input forward

PART V

Retrieval and RAG

Six components that let a model answer from your documents instead of from its training data — loading, splitting, embedding, storing, searching, and generating.

13 Document Loaders

Turn a file, a URL, or a database row into LangChain [Document](#) objects.

Why they exist

Your knowledge lives in PDFs, Word files, Confluence pages, CSV exports and web pages. Each format needs different parsing. Loaders hide that behind one method, and — critically — they attach **metadata** so you can later tell the user where an answer came from.

Basic use

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("notes.txt")
docs = loader.load()

print(len(docs))
print(docs[0].page_content[:200])
print(docs[0].metadata)
```

OUTPUT

```
1
Amazon EC2 provides resizable compute capacity in the cloud...
{'source': 'notes.txt'}
```

The Document object

Everything downstream — splitters, vector stores, retrievers — speaks in [Documents](#). It has exactly two attributes:

Attribute	Type	Contains
<code>page_content</code>	<code>str</code>	The actual text
<code>metadata</code>	<code>dict</code>	Source path, page number, and anything you add

Metadata is not optional

The single most common complaint from users of an internal AI assistant is "I do not trust it, it will not tell me where that came from." Metadata is your answer. Populate `source`, `page`, `department`, `last_updated` and anything else you can — it costs nothing at ingest and it also gives you filtered retrieval later.

The loaders you will use

Loader	Handles	Install
<code>TextLoader</code>	Plain <code>.txt</code> , <code>.md</code>	built in
<code>PyPDFLoader</code>	PDFs, one Document per page	<code>pypdf</code>

Loader	Handles	Install
<code>Docx2txtLoader</code>	Word <code>.docx</code>	<code>docx2txt</code>
<code>CSVLoader</code>	CSV, one Document per row	built in
<code>JSONLoader</code>	JSON with a jq-style selector	<code>jq</code>
<code>WebBaseLoader</code>	Web pages	<code>beautifulsoup4</code>
<code>DirectoryLoader</code>	A whole folder, recursively	built in
<code>UnstructuredFileLoader</code>	Almost anything else	<code>unstructured</code>

Loading a folder of PDFs

```
from langchain_community.document_loaders import (
    DirectoryLoader, PyPDFLoader
)

loader = DirectoryLoader(
    "./company_docs",
    glob="**/*.pdf",
    loader_cls=PyPDFLoader,
    show_progress=True
)

docs = loader.load()
print(f"Loaded {len(docs)} pages")
```

`PyPDFLoader` produces one Document per page and records the page number in metadata — which is what lets your answer say "policy handbook, page 14".

Adding your own metadata

```
for d in docs:
    d.metadata["department"] = "hr"
    d.metadata["ingested_on"] = "2026-08-05"
    d.metadata["access_level"] = "internal"
```

Later you can restrict retrieval to a department, or filter out documents the current user is not cleared to see. Do this at ingest; retrofitting is painful.

Methods

Method	Behaviour	When
<code>load()</code>	Everything into memory at once	Under a few hundred files
<code>lazy_load()</code>	Yields Documents one at a time	Large corpora — keeps memory flat
<code>aload()</code>	Async load	Web loaders fetching many URLs

14 Text Splitters

Cut long documents into chunks small enough to embed and to fit in a prompt.

Why they exist

Two hard constraints force this:

- 12. Context windows are finite.** A 300-page manual will not fit in a prompt, and even where it fits, sending it on every question is ruinously expensive.
- 13. Retrieval precision collapses at length.** Embed an entire chapter as one vector and its meaning averages out into mush. Ask a specific question and nothing matches well. Small, focused chunks retrieve sharply.

The default splitter

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_documents(docs)
print(len(chunks))
print(chunks[0].page_content)
```

Why "recursive"

It tries a list of separators in order and only falls back when a chunk is still too big:

- | | | | |
|----|--------|---------------------|----------------|
| 1. | "\n\n" | split on paragraphs | <- preferred |
| 2. | "\n" | split on lines | |
| 3. | " " | split on words | |
| 4. | " | split on characters | <- last resort |

Figure 14.1 — Separator fallback order. It cuts at the largest natural boundary that fits.

That ordering is the whole point. A naive character splitter chops mid-sentence and mid-word; the recursive splitter preserves paragraphs wherever it can, so chunks stay semantically whole.

chunk_overlap, and why it is not waste

Overlap repeats the tail of one chunk at the head of the next. It looks like duplication. It is insurance: without it, a definition that straddles a boundary is destroyed by the cut, and no query will ever retrieve it intact.

```
Chunk 1: [.....]
Chunk 2:      [.....]
              ^-----^
              overlap
```

Figure 14.2 — Overlap keeps boundary-straddling ideas retrievable.

Choosing the numbers

Content type	chunk_size	overlap	Reasoning
FAQ, chat logs, Q&A	300 – 500	50	Each entry is already self-contained
Policy docs, manuals	800 – 1000	100 – 150	Arguments run across paragraphs
Legal contracts	1000 – 1500	200	Clauses reference one another
Source code	1000 – 2000	100	Split on function boundaries instead
Research papers	1000	200	Dense prose, long dependencies

The most common tuning mistake

Chunks that are too large. When retrieval quality is poor, the instinct is to retrieve **more** text; the fix is almost always **smaller, sharper** chunks and a higher [k](#). Start at 500/50, measure, and only then adjust.

Specialised splitters

```
# Count real tokens, not characters – matches the model's limit exactly
splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    chunk_size=500, chunk_overlap=50
)

# Respect Markdown structure
from langchain_text_splitters import MarkdownHeaderTextSplitter
md = MarkdownHeaderTextSplitter(
    headers_to_split_on=[("#", "h1"), ("##", "h2"), ("###", "h3")]
)

# Split code on syntactic boundaries
from langchain_text_splitters import Language
py = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON, chunk_size=1000, chunk_overlap=100
)
```

[MarkdownHeaderTextSplitter](#) promotes headings into metadata, so a retrieved chunk knows which section it came from. For documentation sites, that alone lifts answer quality noticeably.

Methods

Method	Input	Output
<code>split_documents(docs)</code>	List of Documents	List of Documents, metadata preserved
<code>split_text(str)</code>	A raw string	List of strings
<code>create_documents([str])</code>	List of strings	List of Documents

Use `split_documents()`, not `split_text()`

`split_text()` throws your metadata away. Once the source path and page number are gone you cannot cite anything, and you cannot filter retrieval. Always split Documents, not strings.

15 Embeddings

Convert text into a list of numbers that encodes its meaning.

Why they exist

Keyword search fails on the way people actually ask questions. Someone types "how do I fire someone" and your handbook says "involuntary termination procedure". Zero words in common; keyword search returns nothing.

Embeddings solve that. Each piece of text becomes a point in a high-dimensional space, positioned by **meaning**. Texts that mean similar things land near each other, whatever words they used. Search becomes geometry.

Basic use

```
from langchain_openai import OpenAIEmbeddings

embedding = OpenAIEmbeddings(model="text-embedding-3-small")

vector = embedding.embed_query("Artificial Intelligence")
print(len(vector))
print(vector[:5])
```

OUTPUT

```
1536
[-0.0134, 0.0217, -0.0089, 0.0342, -0.0011]
```

1,536 numbers. Individually meaningless. Collectively, a coordinate for the concept.

Similarity is an angle

Closeness is measured with **cosine similarity** — the cosine of the angle between two vectors, running from -1 to 1.

```
import numpy as np

def cosine(a, b):
    a, b = np.array(a), np.array(b)
    return float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b)))

v1 = embedding.embed_query("How do I terminate an employee?")
v2 = embedding.embed_query("Involuntary termination procedure")
v3 = embedding.embed_query("Best biryani in Hyderabad")

print(cosine(v1, v2))
print(cosine(v1, v3))
```

OUTPUT

```
0.81
0.06
```

0.81 for two phrases sharing barely a word. 0.06 for the unrelated one. That gap is the entire mechanism behind every RAG system you will build.

Cosine score	Interpretation
0.90 – 1.00	Near-duplicate — often the same sentence reworded
0.75 – 0.90	Strongly related — good retrieval hit
0.50 – 0.75	Loosely related — usable with reranking
below 0.50	Unrelated — should not be in your context window

Two methods, and the difference matters

Method	Input	Returns	Used for
<code>embed_query(str)</code>	One string	One vector	The user's question, at search time
<code>embed_documents([str])</code>	List of strings	List of vectors	Your corpus, at ingest time

They are not interchangeable

Some embedding models are asymmetric — they prepend different internal instructions for a short question than for a long passage. Calling the wrong one degrades retrieval in a way that is very hard to diagnose. Questions go through `embed_query`. Corpus goes through `embed_documents`. Always.

Choosing a model

Model	Dimensions	Notes
<code>text-embedding-3-small</code>	1536	The sensible default — cheap and strong
<code>text-embedding-3-large</code>	3072	Higher accuracy, ~6x the cost and storage
<code>models/embedding-001</code> (Google)	768	Good multilingual coverage
<code>all-MiniLM-L6-v2</code> (local)	384	Runs on CPU, free, no data leaves your network
<code>BAAI/bge-m3</code> (local)	1024	Strong open multilingual option

The rule you must not break

Index and query with the **same embedding model**. Different models produce vectors in incompatible spaces — the numbers are meaningless across them. If you change models, you must re-embed the entire corpus. Record the model name in your index metadata so future-you knows.

Cost, in practice

Embedding is cheap and one-off; generation is expensive and per-request. A 500-page corpus costs cents to embed. Cache aggressively anyway — re-embedding an unchanged document on every deploy is pure waste.

```
from langchain.embeddings import CacheBackedEmbeddings
from langchain.storage import LocalFileStore

store = LocalFileStore("./embedding_cache/")
cached = CacheBackedEmbeddings.from_bytes_store(
    embedding, store, namespace=embedding.model
)
```

16 Vector Stores

The database that holds your embeddings and finds the nearest ones fast.

Why they exist

With 50,000 chunks, comparing a query against every vector one at a time is far too slow for a web request. Vector stores use approximate-nearest-neighbour indexes (HNSW, IVF) to return the top matches in milliseconds, and they store the original text and metadata alongside so you get usable Documents back, not bare numbers.

Building one with FAISS

```
from langchain_community.vectorstores import FAISS

db = FAISS.from_documents(chunks, embedding)
```

That one line embeds every chunk, builds the index, and keeps the text and metadata attached. It is the highest work-to-code ratio in the entire library.

Searching directly

```
results = db.similarity_search("What is EC2?", k=3)

for d in results:
    print(d.metadata["source"], "->", d.page_content[:80])
```

Searching with scores

```
for doc, score in db.similarity_search_with_score("What is EC2?", k=3):
    print(round(score, 3), doc.page_content[:60])
```

Scores let you set a floor. If the best match is weak, do not answer from it — say "I could not find that in the documentation." A system that admits ignorance is trusted; one that confabulates is switched off.

Persisting the index

```
db.save_local("faiss_index")

db = FAISS.load_local(
    "faiss_index",
    embedding,
    allow_dangerous_deserialization=True
)
```

That flag is a real warning

FAISS indexes are pickled. Loading one executes code. Only ever set `allow_dangerous_deserialization=True` for an index **you** created. Never load an index downloaded

from the internet or supplied by a customer.

Incremental updates

```
db.add_documents(new_chunks)          # append
db.delete(ids=["doc_id_1", "doc_id_2"]) # remove
db.save_local("faiss_index")          # persist
```

You almost never rebuild from scratch in production. New policy published? Add its chunks. Policy withdrawn? Delete them, or your assistant will keep quoting a document that no longer exists — a genuine compliance problem.

Metadata filtering

```
db.similarity_search(
    "leave policy",
    k=4,
    filter={"department": "hr", "access_level": "internal"}
)
```

Filtering restricts the search *before* similarity is computed. This is how multi-tenant systems keep one customer's data out of another customer's answers — and it is why Chapter 13 insisted on populating metadata.

Which store to choose

Store	Runs	Good for	Watch out
FAISS	In process	Prototypes, small/medium corpora	No server; you manage the file
Chroma	Local or server	Development with persistence	Modest scale ceiling
Pinecone	Managed cloud	Production, zero ops	Per-month cost; data leaves your VPC
Qdrant	Self-host or cloud	Rich filtering, open source	You run the cluster
pgvector	PostgreSQL	Teams already on Postgres	Slower at very large scale
Milvus / Weaviate	Cluster	Hundreds of millions of vectors	Real operational overhead

Start with FAISS

Prove the pipeline works with FAISS on your laptop. Migrating later is a two-line change, because the retriever interface is identical across every store. Do not begin a project by provisioning a vector database cluster.

Methods

Method	Purpose
<code>from_documents(docs, embedding)</code>	Build a new index in one call
<code>add_documents(docs)</code>	Append to an existing index
<code>similarity_search(q, k)</code>	Top-k nearest Documents
<code>similarity_search_with_score(q, k)</code>	Same, with distance scores
<code>max_marginal_relevance_search(q, k)</code>	Top-k, penalised for redundancy
<code>delete(ids)</code>	Remove entries
<code>save_local() / load_local()</code>	Persist and reload (FAISS)
<code>as_retriever()</code>	Wrap as a Runnable for chains

17 Retrievers

A thin Runnable wrapper over search: a question goes in, a list of Documents comes out.

Why they exist

`similarity_search()` is a vector-store method. A retriever is a **Runnable**, so it can be piped. That is the whole difference, and it is what lets retrieval sit inside an LCEL chain rather than in imperative code around it.

It also decouples the chain from the storage. A retriever might be backed by FAISS, Pinecone, Elasticsearch, a SQL query, or a web API — the chain neither knows nor cares.

Basic use

```
retriever = db.as_retriever()

docs = retriever.invoke("What is Kubernetes?")
print(len(docs))
print(docs[0].page_content[:150])
```

Configuring it

```
retriever = db.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4}
)
```

Search types

Type	Behaviour	Use when
<code>similarity</code>	Plain top-k nearest	Default; almost always fine
<code>mmr</code>	Top-k, penalised for redundancy	Chunks overlap heavily and you keep getting near-duplicates
<code>similarity_score_threshold</code>	Only results above a floor	You would rather return nothing than something weak

```
# MMR – diverse results
retriever = db.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 4, "fetch_k": 20, "lambda_mult": 0.5}
)

# Threshold – refuse weak matches
retriever = db.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"score_threshold": 0.5, "k": 4}
)
```

Choosing k

k	Effect
1 – 2	Very precise; misses context spread across chunks
3 – 5	The sweet spot for most applications
6 – 10	Broad questions and summaries; more tokens, more cost
above 10	Usually a sign your chunks are too small or your embeddings are wrong

Retrievers worth knowing

Retriever	What it adds
<code>VectorStoreRetriever</code>	The default — semantic search
<code>BM25Retriever</code>	Classic keyword search; excellent for exact codes and names
<code>EnsembleRetriever</code>	Blends semantic and keyword results — usually the best single upgrade
<code>MultiQueryRetriever</code>	Rewrites the question several ways, merges the hits
<code>ContextualCompressionRetriever</code>	Reranks and trims retrieved text before it reaches the prompt
<code>ParentDocumentRetriever</code>	Searches small chunks, returns their larger parent for context

HYBRID SEARCH — THE HIGHEST-VALUE 8 LINES IN RAG

```
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever

bm25 = BM25Retriever.from_documents(chunks)
bm25.k = 4

hybrid = EnsembleRetriever(
    retrievers=[bm25, db.as_retriever(search_kwargs={"k": 4})],
    weights=[0.4, 0.6]
)
```

Why hybrid wins

Semantic search understands meaning but fumbles exact tokens — error code [E5023](#), part number [AX-7741](#), a person's surname. BM25 nails those and understands nothing else. Ensemble them and each covers the other's blind spot. If your RAG system is underperforming, try this before anything more exotic.

18 RetrievalQA — the Classic Chain

A pre-built chain that does retrieval and generation in one call.

The classic form

```
from langchain.chains import RetrievalQA

qa = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever
)

print(qa.invoke("Explain Docker"))
```

OUTPUT

```
{'query': 'Explain Docker',
 'result': 'Docker is a containerisation platform that...'}

```

Under the hood it does exactly what you would do by hand:

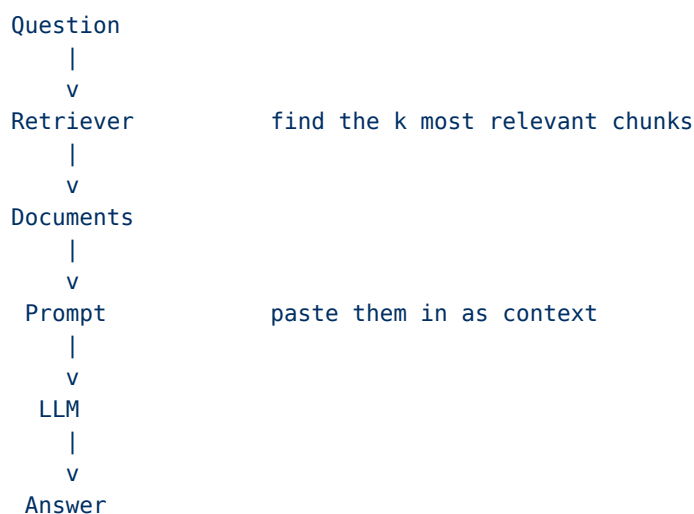


Figure 18.1 — RetrievalQA in four steps.

Returning sources

```
qa = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True
)

result = qa.invoke("What is our leave policy?")
print(result["result"])

for d in result["source_documents"]:
```

```
print("-", d.metadata.get("source"), "page", d.metadata.get("page"))
```

The chain_type argument

Type	Strategy	Use when
<code>stuff</code>	Put every chunk in one prompt	Default. Fast, cheap, one call.
<code>map_reduce</code>	Summarise each chunk, then combine	Retrieved text exceeds the context window
<code>refine</code>	Build the answer chunk by chunk	Long analytical answers over many sources
<code>map_rerank</code>	Answer per chunk, return the best	A single factual answer buried in one chunk

The modern replacement

Know it, but write the LCEL version

`RetrievalQA` is legacy. It works, and you will meet it in every existing codebase, so you must be able to read it. New code should use the explicit LCEL chain below — it is the same logic with none of the hidden behaviour, and you can change any single step without fighting the abstraction.

THE CHAIN TO WRITE IN 2026

```
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("""
Answer the question using only the context below.
If the context does not contain the answer, say you do not know.

Context:
{context}

Question: {question}
""")

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs,
     "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

print(rag_chain.invoke("Explain Docker"))
```

Everything is visible: which prompt, which retriever, what the model receives. Nothing is hidden inside a factory method. And because it is plain LCEL, it streams, batches and traces like any other chain.

The instruction that prevents hallucination

Note the second line of that prompt: `"If the context does not contain the answer, say you do not know."` Without it, a model handed irrelevant context will cheerfully answer from training data and present it as if it came from your documents. That one sentence is the difference between a trustworthy internal assistant and a liability.

19 The End-to-End RAG Pipeline

Every component from Part V, assembled into one working program.

The complete script

RAG.PY — COMPLETE AND RUNNABLE

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS

# 1. Model
llm = ChatOpenAI(model="gpt-4.1", temperature=0)

# 2. Load
docs = TextLoader("aws.txt").load()

# 3. Split
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
chunks = splitter.split_documents(docs)

# 4. Embed
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# 5. Store
db = FAISS.from_documents(chunks, embeddings)

# 6. Retrieve
retriever = db.as_retriever(search_kwargs={"k": 4})

# 7. Prompt
prompt = ChatPromptTemplate.from_template("""
Answer the question using only the context below.
If the answer is not in the context, say you do not know.

Context:
{context}

Question:
{question}
""")

# 8. Helper
def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

# 9. Chain
rag_chain = (
    {"context": retriever | format_docs,
```

```

        "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

# 10. Ask
print(rag_chain.invoke("What is Amazon EC2?"))

```

The two phases

Steps 2 to 5 are **ingestion**. They run once, offline, when documents change. Steps 6 to 10 are **query**. They run on every user request.

Do not re-ingest on every request

The most common beginner architecture mistake is putting the loading, splitting and embedding inside the request handler. Every question then re-embeds the entire corpus — slow, expensive, and pointless. Ingest in a scheduled job, persist the index with `save_local()`, and have the API load it once at startup.

Splitting the two phases properly

INGEST.PY

```

# ingest.py – runs nightly, or on document change
chunks = splitter.split_documents(DirectoryLoader(...).load())
FAISS.from_documents(chunks, embeddings).save_local("index")

```

APP.PY

```

# app.py – runs on startup, serves every request
db = FAISS.load_local("index", embeddings,
                    allow_dangerous_deserialization=True)
retriever = db.as_retriever(search_kwargs={"k": 4})
rag_chain = ({ "context": retriever | format_docs,
               "question": RunnablePassthrough()
               | prompt | llm | StrOutputParser() })

@app.get("/ask")
def ask(q: str):
    return {"answer": rag_chain.invoke(q)}

```

Returning the answer with its citations

```

from langchain_core.runnables import RunnableParallel

rag_with_sources = RunnableParallel(
    context=retriever,
    question=RunnablePassthrough()
).assign(
    answer=(

```

```

        RunnablePassthrough.assign(
            context=lambda x: format_docs(x["context"]))
        | prompt | llm | StrOutputParser()
    )

result = rag_with_sources.invoke("What is Amazon EC2?")
print(result["answer"])
for d in result["context"]:
    print("source:", d.metadata.get("source"))

```

When answers are poor — the diagnostic order

Resist the urge to change the model first. It is almost never the model.

Symptom	Likely cause	Fix
Retrieved chunks are irrelevant	Chunks too large, or wrong embedding model	Halve <code>chunk_size</code> ; verify same model at index and query
Right document, wrong passage	Boundary cut through the answer	Raise <code>chunk_overlap</code>
Exact codes and IDs never found	Semantic search cannot do exact tokens	Add BM25 via <code>EnsembleRetriever</code>
Answer invents facts	Prompt permits it	Add the "say you do not know" instruction; add a score threshold
Repetitive, near-identical chunks	Redundant retrieval	Switch to <code>search_type="mmr"</code>
Slow responses	Re-ingesting per request, or <code>k</code> too high	Persist the index; lower <code>k</code>

Evaluate before you ship

Write twenty questions you know the answers to. Run them after every change. Track two numbers: did the right chunk get retrieved, and was the final answer correct. Without that baseline, every "improvement" you make is a guess — and half of them will make things worse.

PART VI

Tools and Agents

Giving the model the ability to act — to call your functions, query your systems, and decide for itself what to do next.

20 Tools

Python functions the model is allowed to call.

Why they exist

A model cannot look up today's exchange rate, query your orders table, send an email, or reliably multiply two large numbers. Tools bridge that gap: you expose a function, the model decides when it is needed, and your code runs it.

The decorator

```
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b
```

The docstring is the interface

The model never sees your function body. It sees the name, the type hints, and the docstring — and decides from those alone whether to call it. A vague docstring produces a tool that is never used, or used wrongly. Write it as an instruction to a colleague who cannot read the code.

A well-specified tool

```
from langchain.tools import tool

@tool
def get_order_status(order_id: str) -> str:
    """Look up the current status of a customer order.

    Use this whenever a customer asks where their order is,
    whether it has shipped, or when it will arrive.

    Args:
        order_id: The order reference, e.g. 'ORD-48213'.

    Returns:
        A short status line including the courier and ETA.
    """
    row = db.query("SELECT status, courier, eta FROM orders "
                  "WHERE id = %s", order_id)
    if not row:
        return f"No order found with id {order_id}."
    return f"{row.status} via {row.courier}, expected {row.eta}."
```

Note the failure path returns a **sentence**, not an exception. The model reads tool output as text and can recover from "no order found" by asking the user to check the number. An unhandled exception just crashes the loop.

Inspecting a tool

```
print(add.name)           # add
print(add.description)    # Add two numbers together.
print(add.args)           # {'a': {'type': 'integer'}, ...}
```

Explicit schemas

```
from pydantic import BaseModel, Field
from langchain.tools import tool

class SearchInput(BaseModel):
    query: str = Field(description="Search terms")
    max_results: int = Field(default=5, description="How many to return, 1-20")

@tool(args_schema=SearchInput)
def search_docs(query: str, max_results: int = 5) -> str:
    """Search the internal documentation."""
    ...
```

Rules for tools that behave

- 14. One job per tool.** A tool that does three things will be called for the wrong one.
- 15. Return a string, or something trivially stringifiable.** The output goes back into the prompt.
- 16. Handle your own errors.** Return the problem as text; never let an exception escape.
- 17. Keep output short.** A tool returning 400 rows of JSON will blow the context window and cost a fortune.
- 18. Guard destructive actions.** A tool that deletes, refunds or emails should require confirmation or run against a queue a human approves.

Security

A tool is code the model can trigger, and the model can be influenced by text from a user or from a retrieved document. Never build a tool that executes arbitrary SQL or shell commands from model input. Parameterise everything, whitelist operations, and scope database credentials to read-only wherever you can.

21 Tool Calling

Attaching tools to a model so it can request them.

bind_tools()

```
llm_with_tools = llm.bind_tools([add, get_order_status])
```

This does not run anything. It tells the model what is available, by sending the tool schemas along with every request.

What comes back

```
response = llm_with_tools.invoke("What is 2 + 10?")

print(response.content)      # often empty
print(response.tool_calls)
```

OUTPUT

```
''
[{'name': 'add',
  'args': {'a': 2, 'b': 10},
  'id': 'call_x7Kd92',
  'type': 'tool_call'}]
```

The model does not execute anything

It returns a **request**: this tool, these arguments. Your code runs it. That separation is deliberate and it is the security boundary — you decide what actually gets executed, always.

The manual loop

Understanding this by hand is what makes agents stop being magic:

```
from langchain_core.messages import HumanMessage, ToolMessage

tools = {"add": add, "get_order_status": get_order_status}
messages = [HumanMessage(content="What is 2 + 10?")]

# 1. Ask the model
ai_msg = llm_with_tools.invoke(messages)
messages.append(ai_msg)

# 2. Run whatever it asked for
for call in ai_msg.tool_calls:
    result = tools[call["name"]].invoke(call["args"])
    messages.append(
        ToolMessage(content=str(result), tool_call_id=call["id"])
    )
```

```
# 3. Send the results back so it can answer
final = llm_with_tools.invoke(messages)
print(final.content)
```

OUTPUT

```
2 + 10 equals 12.
```

Three model calls' worth of state in one list. The `tool_call_id` matching is what tells the model which result belongs to which request — get it wrong and the model gets confused about its own history.

Parallel tool calls

Modern models will request several tools at once when the tasks are independent. `tool_calls` is a list precisely because of that — always iterate it, never assume there is exactly one.

Forcing a choice

```
# Must call this specific tool
llm.bind_tools([add], tool_choice="add")

# Must call at least one tool
llm.bind_tools(tools, tool_choice="any")

# Model decides (default)
llm.bind_tools(tools, tool_choice="auto")
```

22 Agents

A model in a loop, with tools, deciding its own next step.

The reasoning cycle

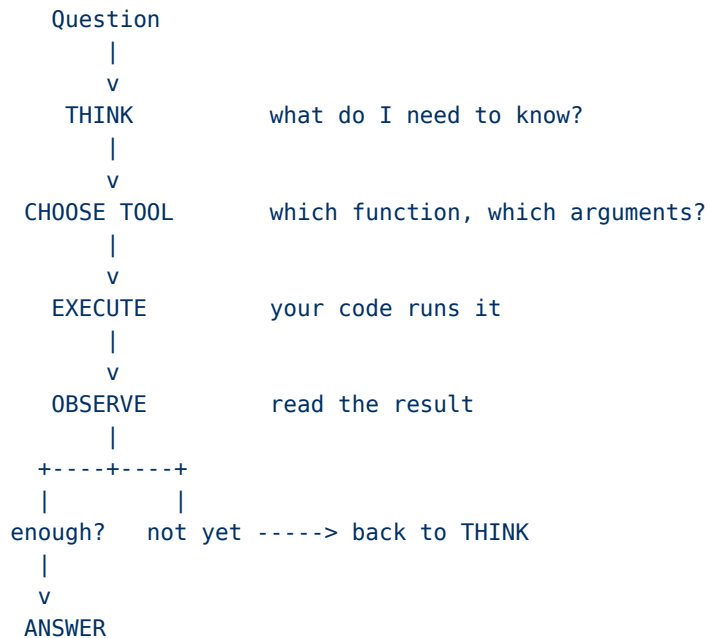


Figure 22.1 — Think, act, observe, repeat. Loop until the question can be answered.

What makes it an agent

A chain has a fixed path decided by you at build time. An agent decides the path at runtime. Same components, different control flow — and that difference is the whole reason agents are both powerful and harder to operate.

	Chain	Agent
Steps	Fixed by you	Chosen by the model
Predictability	High	Lower
Cost per request	Known	Varies with iterations
Debugging	Straightforward	Requires tracing
Right for	A defined workflow	An open-ended goal

Use the simplest thing that works

Agents are fashionable and frequently unnecessary. If you know the steps — retrieve, then answer — write a chain. Reach for an agent only when the sequence genuinely cannot be known in advance. A chain that always works beats an agent that usually does.

Building one

```
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant with access to tools. "
              "Use them when they help. Do not guess at facts "
              "you could look up."),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad")
])

agent = create_tool_calling_agent(llm, tools, prompt)
```

agent_scratchpad is mandatory

That placeholder is where the loop's working memory lives — every tool the agent has already called and every result it got back. Omit it and the agent forgets what it just did and calls the same tool forever. It is the most common agent bug there is.

23 AgentExecutor

The runtime that drives the loop.

Why it exists

`create_tool_calling_agent` builds the decision-maker. It does not run anything. `AgentExecutor` is the engine: it calls the agent, executes the requested tools, appends the results to the scratchpad, calls the agent again, and stops when the agent produces a final answer — or when a limit is hit.

Running it

```
from langchain.agents import AgentExecutor

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True
)

result = agent_executor.invoke({"input": "2 + 10"})
print(result["output"])
```

OUTPUT

```
> Entering new AgentExecutor chain...
Invoking: `add` with {'a': 2, 'b': 10}
12
2 + 10 equals 12.
> Finished chain.
```

The settings that keep it safe

```
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True,
    max_iterations=6,
    max_execution_time=45,
    early_stopping_method="force",
    handle_parsing_errors=True,
    return_intermediate_steps=True
)
```

Setting	Why it matters
<code>max_iterations</code>	Hard cap on loop count. Without it a confused agent can loop until your bill notices.
<code>max_execution_time</code>	Wall-clock ceiling in seconds. Protects your request timeout.
<code>handle_parsing_errors</code>	Feeds a malformed response back for a retry instead of raising

Setting	Why it matters
<code>return_intermediate_steps</code>	Returns every tool call and result — essential for debugging and audit
<code>verbose</code>	Prints the reasoning trace. On in development, off in production.

Set `max_iterations` before you set anything else

An agent given a task it cannot complete will retry indefinitely — every iteration a paid API call. This is the single most common way teams get an unpleasant surprise on their first invoice. Six is a reasonable default for most tools.

Inspecting what it did

```
result = agent_executor.invoke({"input": "Where is order ORD-48213?"})

for action, observation in result["intermediate_steps"]:
    print("TOOL :", action.tool)
    print("ARGS :", action.tool_input)
    print("RESULT:", observation)
    print("-" * 50)
```

In regulated environments this is not a debugging convenience — it is the audit trail you will be asked to produce.

A complete agent

AGENT.PY — COMPLETE AND RUNNABLE

```
from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

llm = ChatOpenAI(model="gpt-4.1", temperature=0)

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

tools = [add, multiply]

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a precise calculator assistant. "
              "Always use the tools rather than computing yourself."),
    ("human", "{input}"),
```

```
    MessagesPlaceholder(variable_name="agent_scratchpad")
  ])

agent = create_tool_calling_agent(llm, tools, prompt)

executor = AgentExecutor(
    agent=agent, tools=tools,
    verbose=True, max_iterations=5
)

print(executor.invoke({"input": "What is (12 + 8) times 4?"})["output"])
```

OUTPUT

```
Invoking: `add` with {'a': 12, 'b': 8} -> 20
Invoking: `multiply` with {'a': 20, 'b': 4} -> 80
(12 + 8) times 4 equals 80.
```

Two tools, two iterations, correct answer. The model worked out the order of operations on its own — nobody wrote that sequence.

Where agents go next

[AgentExecutor](#) runs one fixed loop shape. Once you need branching, human approval steps, parallel sub-agents, retries with different strategies, or the ability to pause and resume a workflow, you have outgrown it. That is what **LangGraph** is for, and it is the natural next thing to learn after this handbook.

PART VII

Memory and Conversation State

Models remember nothing between calls. Everything that feels like memory is something your application chose to resend.

24 Conversation Memory — Legacy and Modern

How LangChain used to manage conversation state, and how it does now.

The legacy classes

Older LangChain used dedicated memory objects that a chain would read from and write to automatically:

LEGACY

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory()

memory.save_context(
    {"input": "Hi"},
    {"output": "Hello"}
)

print(memory.load_memory_variables({}))
```

OUTPUT

```
{'history': 'Human: Hi\nAI: Hello'}
```

Method	Purpose
<code>save_context(inputs, outputs)</code>	Record one exchange
<code>load_memory_variables({})</code>	Retrieve the accumulated history
<code>clear()</code>	Wipe the conversation

The family of legacy memory types

Class	Strategy	Trade-off
<code>ConversationBufferMemory</code>	Keep everything	Simple; eventually overflows the context window
<code>ConversationBufferWindowMemory</code>	Keep the last k turns	Bounded; forgets the beginning
<code>ConversationSummaryMemory</code>	Summarise older turns	Compact; costs an extra call and loses detail
<code>ConversationSummaryBufferMemory</code>	Recent verbatim + older summarised	Best of both; most complex
<code>ConversationTokenBufferMemory</code>	Keep within a token budget	Precise cost control

Why they were retired

Memory was implicit. State mutated inside the chain, which made it invisible in traces, awkward to persist, impossible to branch, and a poor fit for the stateless request handlers most applications are built from. **You will still find these classes in existing projects, so you need to read them — but do not choose them for new work.**

The modern approach

State is now explicit. You own the message list; you decide what to keep; you decide where it is stored. Nothing hides.

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

store = {}

def get_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

chain_with_history = RunnableWithMessageHistory(
    chain,
    get_history,
    input_messages_key="question",
    history_messages_key="history"
)

chain_with_history.invoke(
    {"question": "My name is Sreekanth"},
    config={"configurable": {"session_id": "user-42"}}
)

chain_with_history.invoke(
    {"question": "What is my name?"},
    config={"configurable": {"session_id": "user-42"}}
)
```

OUTPUT

Your name is Sreekanth.

The `session_id` is what keeps two users apart. In production `get_history` reads from Redis or Postgres instead of a dictionary, so state survives a restart and works across multiple server processes.

Persistent history

```
from langchain_community.chat_message_histories import (
    RedisChatMessageHistory
)

def get_history(session_id: str):
    return RedisChatMessageHistory(
```

```
        session_id, url="redis://localhost:6379"  
    )
```

Trimming, because context is finite

```
from langchain_core.messages import trim_messages  
  
trimmer = trim_messages(  
    max_tokens=2000,  
    strategy="last",  
    token_counter=llm,  
    include_system=True,  
    start_on="human"  
)  
  
chain = trimmer | prompt | llm | StrOutputParser()
```

`include_system=True` keeps the system message even when older turns are dropped — otherwise a long conversation quietly loses its persona and its rules halfway through.

25 Message History in Practice

The mechanics underneath every chatbot you have ever used.

The whole trick

```
from langchain_core.messages import HumanMessage

history = [
    HumanMessage(content="My name is Sreekanth"),
    HumanMessage(content="What is my name?")
]

response = llm.invoke(history)
print(response.content)
```

OUTPUT

Your name is Sreekanth.

The model "remembered" nothing. You resent the first message alongside the second. That is the entire mechanism — there is no hidden state anywhere.

A minimal working chatbot

CHATBOT.PY — COMPLETE AND RUNNABLE

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import (
    SystemMessage, HumanMessage, AIMessage
)

llm = ChatOpenAI(model="gpt-4.1", temperature=0.3)

messages = [
    SystemMessage(content="You are a friendly DevOps trainer at "
                          "Cloud Soft Solutions. Keep answers short.")
]

while True:
    user = input("You: ")
    if user.lower() in {"exit", "quit"}:
        break

    messages.append(HumanMessage(content=user))
    reply = llm.invoke(messages)
    messages.append(AIMessage(content=reply.content))

    print("Bot:", reply.content)
```

Twenty lines. The two `append` calls are the entire memory system. Delete either one and the bot develops amnesia.

Why you cannot just keep appending forever

- **Context limits.** Every model has a ceiling. Cross it and the request fails outright.
- **Cost.** You pay for input tokens on every turn. A 50-turn conversation resends 49 turns of history each time — cost grows quadratically with conversation length.
- **Latency.** Bigger prompts take longer to process.
- **Attention dilution.** Models attend less reliably to the middle of very long inputs. More history can mean worse answers.

Four strategies

Strategy	How	Best for
Keep everything	Append forever	Short sessions only — support chats, demos
Sliding window	Keep the last N turns	Most chatbots. Simple, predictable cost.
Summarise older turns	Compress the head, keep the tail verbatim	Long advisory or tutoring sessions
Retrieve from history	Embed past turns; retrieve only relevant ones	Assistants with months of history

The sliding window, written out

```
MAX_TURNS = 10

def trim(messages):
    system = messages[0]
    rest = messages[1:]
    # 2 messages per turn (human + AI)
    return [system] + rest[-(MAX_TURNS * 2):]

messages = trim(messages)
```

Keep the system message pinned, drop the oldest exchanges. Ninety percent of production chatbots do exactly this and nothing more.

One session per user — always

The bug that ends careers

A module-level `messages` list is shared across every request your server handles. Two users talking at once will see each other's conversation. Key history by session or user id, store it outside the process, and never hold conversation state in a global.

PART VIII

Reference and Next Steps

The production class table, a study plan, a troubleshooting guide, and where the road goes after this book.

26 Production Class Reference

The complete table. Print it, pin it to your monitor, and stop searching the documentation for the same six method names.

Class	Purpose	Common methods	Real-world use
<code>ChatOpenAI</code>	Call an LLM	<code>invoke()</code> , <code>batch()</code> , <code>stream()</code> , <code>bind_tools()</code>	Chatbots, copilots
<code>ChatPromptTemplate</code>	Reusable prompts	<code>from_template()</code> , <code>from_messages()</code> , <code>invoke()</code> , <code>partial()</code>	Prompt engineering
<code>SystemMessage</code>	Standing instructions	Constructor	Persona and rules
<code>HumanMessage</code>	User turn	Constructor	Multi-turn conversation
<code>AIMessage</code>	Assistant turn	Constructor, <code>.content</code>	Replaying history
<code>StrOutputParser</code>	Message → string	<code>invoke()</code>	Chat responses
<code>JsonOutputParser</code>	Text → dict	<code>invoke()</code> , <code>get_format_instructions()</code>	APIs, automation
<code>with_structured_output()</code>	Typed objects	Called on the model	Extraction, HR, CRM, invoices
<code>RunnableLambda</code>	Wrap Python functions	<code>invoke()</code> , <code>batch()</code>	Validation, preprocessing
<code>RunnableParallel</code>	Concurrent branches	<code>invoke()</code>	Summary + keywords + sentiment
<code>RunnablePassthrough</code>	Forward input unchanged	<code>invoke()</code> , <code>assign()</code>	RAG wiring
<code>RunnableBranch</code>	Conditional routing	<code>invoke()</code>	Intent-based dispatch
<code>TextLoader</code> , <code>PyPDFLoader</code> , <code>CSVLoader</code>	Load documents	<code>load()</code> , <code>lazy_load()</code>	RAG ingestion
<code>RecursiveCharacterTextSplitter</code>	Chunk documents	<code>split_documents()</code> , <code>split_text()</code>	Large document processing
<code>OpenAIEmbeddings</code>	Text → vectors	<code>embed_query()</code> , <code>embed_documents()</code>	Semantic search
<code>FAISS</code> , <code>Chroma</code>	Store vectors	<code>from_documents()</code> , <code>similarity_search()</code> , <code>save_local()</code>	Vector databases
<code>VectorStoreRetriever</code>	Fetch context	<code>invoke()</code> , <code>batch()</code>	RAG pipelines
<code>EnsembleRetriever</code>	Hybrid search	<code>invoke()</code>	Codes, IDs, proper nouns

Class	Purpose	Common methods	Real-world use
<code>RetrievalQA</code> (legacy)	Retrieval + generation	<code>invoke()</code>	Document Q&A in older code
<code>@tool</code>	Define callable tools	Decorator	External integrations
<code>bind_tools()</code>	Expose tools to a model	Called on the model	Function calling
<code>create_tool_calling_agent</code>	Build an agent	Factory function	Autonomous workflows
<code>AgentExecutor</code>	Run the agent loop	<code>invoke()</code>	AI agents
<code>RunnableWithMessageHistory</code>	Session state	<code>invoke()</code> with <code>session_id</code>	Chat applications
<code>trim_messages</code>	Bound the history	Used as a Runnable	Long conversations

27 A 21-Day Study Plan

Ninety minutes a day. Build, do not read.

Days	Focus	Build this
1 – 2	Setup, ChatOpenAI , invoke/batch/stream	A CLI that streams answers to any question
3 – 4	ChatPromptTemplate , message classes	A configurable "explain X at level Y" tool
5 – 6	Parsers and structured output	A resume parser that outputs a validated Pydantic object
7 – 8	LCEL and the three runnables	A parallel analyser: summary + keywords + sentiment
9 – 10	Loaders and splitters	Ingest a folder of PDFs and print chunk statistics
11 – 12	Embeddings and cosine similarity	A semantic search CLI over your own notes
13 – 14	Vector stores and retrievers	A persisted FAISS index with metadata filtering
15 – 16	The full RAG chain	Document Q&A over your company handbook, with citations
17 – 18	Tools and tool calling	Three tools plus a hand-written tool loop
19 – 20	Agents and AgentExecutor	An agent that combines a retriever tool with an API tool
21	Consolidate	Deploy one project behind FastAPI with streaming

Ten projects, in increasing order of difficulty

19. Streaming CLI chatbot with a persona.
20. Bulk content generator using [batch\(\)](#) over a CSV of topics.
21. Resume-to-JSON extractor using [with_structured_output\(\)](#).
22. Multi-language article publisher using [RunnableParallel](#).
23. Semantic search over your own notes directory.
24. PDF question-answering with page-level citations.
25. Hybrid RAG with [EnsembleRetriever](#) over technical documentation.
26. Support bot that answers from a knowledge base and escalates via a tool.
27. SQL agent that answers business questions against a read-only database.
28. Multi-agent research assistant — then rebuild it in LangGraph and compare.

Where to go after this

Next	Why	When
LangGraph	Stateful, branching, resumable workflows with human-in-the-loop	As soon as AgentExecutor feels restrictive
LangSmith	Tracing, evaluation datasets, regression testing	Before your first production deployment

Next	Why	When
Evaluation	RAGAS and similar — measure retrieval and answer quality	The moment you have real users
Serving	FastAPI, streaming, caching, rate limiting	When a demo becomes a product
Guardrails	PII redaction, injection defence, output validation	Before anything customer-facing

The honest summary

Twenty-five classes. Three methods each. One pipe operator.

Everything in modern LangChain is a Runnable that takes an input, does one job, and returns an output — and everything else is a variation on connecting those together. Once that clicks, the library stops being large and starts being obvious.

Appendix A — Troubleshooting

Error or symptom	Cause	Fix
<code>ImportError: cannot import name ChatOpenAI</code>	Old flat import path	Import from <code>langchain_openai</code> , not <code>langchain</code>
<code>ModuleNotFoundError: langchain_core</code>	Package split not installed	<code>pip install langchain-core</code>
<code>KeyError</code> on prompt invoke	A template variable was not supplied	Check <code>prompt.input_variables</code>
<code>KeyError</code> on a literal JSON brace	Braces read as a variable	Double them: <code>{{</code> and <code>}}</code>
Printing gives <code>AIMessage(...)</code>	Forgot <code>.content</code>	Use <code>.content</code> or add <code>StrOutputParser()</code>
<code>RateLimitError</code> during <code>batch()</code>	Too much concurrency	Set <code>config={"max_concurrency": 5}</code>
Agent loops forever	No iteration cap	Set <code>max_iterations</code> on <code>AgentExecutor</code>
Agent repeats one tool endlessly	<code>agent_scratchpad</code> missing from the prompt	Add the <code>MessagesPlaceholder</code>
Tool is never called	Vague docstring	Rewrite it as an explicit instruction with an example
RAG returns nothing relevant	Chunks too large, or mismatched embedding models	Reduce <code>chunk_size</code> ; re-embed with one model
RAG cannot find an exact code	Semantic search alone	Add BM25 through <code>EnsembleRetriever</code>
Answers invent facts	Prompt does not forbid it	Add "if the context lacks the answer, say you do not know"
Chatbot forgets everything	History not appended or not persisted	Append the <code>AIMessage</code> too; key history by session
Users see each other's chats	Global message list	Store history per session id, outside the process
<code>allow_dangerous_deserialization</code> error	FAISS pickle safety check	Set it <code>True</code> only for indexes you created
Costs far higher than expected	Re-ingesting per request, or unbounded history	Persist the index; trim messages; cap <code>max_tokens</code>

Appendix B — Glossary

Term	Meaning
Agent	A model in a loop that chooses its own tools and steps at runtime
Chain	A fixed sequence of components, decided by the developer
Chunk	A piece of a document, sized to embed and retrieve well
Context window	The maximum tokens a model can accept in one request
Cosine similarity	A -1 to 1 measure of the angle between two vectors; how "close in meaning" is computed
Embedding	A vector of numbers representing the meaning of a piece of text
Grounding	Constraining the model to answer from supplied text rather than training data
Hallucination	A fluent, confident, incorrect answer
Hybrid search	Combining semantic (vector) and keyword (BM25) retrieval
LCEL	LangChain Expression Language — the composition syntax
MMR	Maximal Marginal Relevance — top-k results penalised for redundancy
RAG	Retrieval-Augmented Generation — retrieve relevant text, then generate from it
Runnable	The core interface: invoke , batch , stream , and async variants
Scratchpad	The agent's working memory of tools already called and their results
Temperature	Randomness control; 0 is deterministic-ish, higher is more varied
Token	The unit models read and bill in — roughly ¾ of an English word
Tool	A function the model may request your code to execute
Tool call	The model's request naming a tool and its arguments; your code runs it
Vector store	A database optimised for nearest-neighbour search over embeddings

About Cloud Soft Solutions

Cloud Soft Solutions is an IT training and placement institute based in Ameerpet, Hyderabad, with more than 5,500 alumni placed across the industry.

Our programs

Program	Coverage
APEX	Artificial Intelligence, Generative AI, Cloud and Cyber Security
NEXUS	Cloud, DevOps, Site Reliability Engineering and AIOps

Both programs are delivered as live, instructor-led sessions with hands-on labs, real project work, resume preparation, mock interviews and placement assistance.

Aditya Enclave, 5th Floor, Ameerpet, Hyderabad — beside Ameerpet Metro Station

cloudsoftsol.com

LangChain Essentials 2026 · Revision 1.0 · Prepared by Sreekanth Bathalapalli, Lead Trainer.

Code examples are provided for instructional use. Library APIs evolve — always confirm against the official LangChain documentation for the version you have installed.